

claude-windows / edc346c2-92fc-4ad1-b556-c7ab955a5613

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\edc346c2-92fc-4ad1-b556-c7ab955a5613\subagents\workflows\wf_47347652-5ae\agent-a0d6851393b0602be.jsonl`  
- SHA-256: `ee266b366da1e716cb28f26ac4892426f328f453bdc30897d2bd1ee44c771f42`  
- Source modified: `2026-07-05T06:56:46+00:00`  
- Imported at: `2026-07-05T16:48:22+00:00`  
- project: `wf_47347652-5ae`  
- session_id: `edc346c2-92fc-4ad1-b556-c7ab955a5613`
```

Transcript

1. user (2026-07-05T06:53:50.198Z)

SHARED CONTEXT (verified live this session, 2026-07-05):

```
- Repos (local checkouts = current master):  
  * ENGINE: C:\Users\avidu\Projects\khelesutra-guru\badminton-highlight-indexer (GitHub Khelsutra/badminton-highlight-indexer)  
  * SPA: C:\Users\avidu\Projects\khelesutra-guru\khelesutra (web/src is the Studio SPA; GitHub avidullu/khelesutra)  
- The LIVE hosted control-plane is an OLDER build than local master: capabilities reports engine_version "0.1.0",  
  and its capabilities JSON has NO max_upload_mb / max_part_mb fields (current master DOES expose them, added around  
  khelesutra #153). So distinguish "current master code" (what you can read) from "deployed old-build behavior" and  
  note where they likely diverge.  
- Deployed control-plane config.json (base64-injected at boot) sets ONLY:  
  {"deployment":{"profile":"cloud-serving","compute_target":"cloud_run"},"indexing":{"skip_ai_handoff":true},  
   "storage":{"backend":"gcs","output_root":"gs://khelesutra-rally-serving","input_backend":"gcs",  
    "input_root":"gs://khelesutra-rally-serving/videos"},"security":{"..edge auth..}}  
  Note: it does NOT pin any default/segmenter, and does NOT override security.max_upload_mb or max_part_mb.  
- Live capabilities from the deployed CP: default_segmenter="yolo_hybrid",  
  wasb_weights_present=false,  
  gpu.nvidia_smi_present=false, gemini_key_present=false, deployment.cloud_available=true,  
  profile=cloud-serving.  
- Architecture: the CP (Cloud Run service, 4Gi RAM, no GPU, /tmp is RAM-backed) serves the SPA +  
  dispatches  
  cloud jobs to an L4 GPU Cloud Run JOB "rally-worker" which HAS the WASB weights mounted  
  (WASB_WEIGHTS=gcs/models/...).  
- Config defaults known from code: SecurityConfig.max_upload_mb=4096 (MB whole-file),  
  max_part_mb=95 (per part).  
- The e2e/battery that validated the alpha ALWAYS passed segmenter:"wasb_hybrid" EXPLICITLY in  
  /api/process.
```

ISSUE 2 ? the alpha is scoped/validated as WASB-ONLY (wasb_hybrid), but the deployed control-plane's capabilities reports default_segmenter="yolo_hybrid" and wasb_weights_present=false. QUESTION: when a real tester runs a video through the SPA with compute="cloud", does the dispatched L4 worker job actually run wasb_hybrid (validated) or yolo_hybrid (unvalidated)? Trace ALL of:
1. ENGINE: how default_segmenter is resolved for capabilities (backend/main.py capabilities handler + config)

and WHY it's yolo_hybrid when WASB weights are absent (fallback logic / config default order).

2. SPA: does the process call send an explicit "segmenter"? Read khelsutra/web/src/api/client.ts (processVideo)

and web/src/components/IngestPanel.tsx ? is there a segmenter picker, or does it only send compute? If it omits segmenter, the server default applies.

3. ENGINE dispatch: backend/orchestration.py (_maybe_dispatch_remote / the cloud dispatch) ? does it forward the chosen/default segmenter to the rally-worker job, and what does the WORKER default to (does the worker, which HAS wasb weights, resolve wasb_hybrid, or also default to yolo_hybrid)?

NET: state definitively whether a tester's cloud job runs wasb_hybrid or yolo_hybrid today, and whether this is a DEPLOYED-CONFIG gap (config.json doesn't pin the segmenter) vs a code issue vs a non-issue.

Fix options:

pin indexing.segmenter/default to wasb_hybrid in the CP config, and/or have the SPA send segmenter explicitly.

2. assistant (2026-07-05T06:53:56.978Z)

I'll trace the segmenter resolution across all three components. Let me start by exploring the repo structure and pulling the latest.

3. user (2026-07-05T06:54:01.783Z)

Already up to date.

```
---ENGINE---
__pycache__
artifacts
backend
backend.db
CLAUDE.md
config.json
deploy
docs
eval_baselines
indexer.db
mypy.ini
output
pyproject.toml
pytest.ini
rally_index.db
README.md
requirements.txt
requirements-trackers.txt
ruff.toml
run_all_tests.py
run_eval.py
run_regression.py
scratch
scripts
SESSION_HANDOFF.md
tests
tools
training
```

4. user (2026-07-05T06:54:04.037Z)

Already up to date.

```
---SPA---
deploy
docs
locales
```

NEXT_STEPS.md
node_modules
README.md
site
web

5. assistant (2026-07-05T06:54:06.483Z)

Now let me trace the three components. Let me start with the engine capabilities handler, config, and dispatch logic.

6. user (2026-07-05T06:54:08.484Z)

Found 21 files

C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\orchestration.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\main.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\config\models.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\worker__main__.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\ui\assets\index-Cqr7zJeJ.js
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\pipeline\segmenters\yolo_hybrid.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\pipeline\detectors\base.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\pipeline\segmenters\trajectory_hybrid.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\reporting\harvest.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\pipeline\segmenters\motion.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\pipeline\segmenters\wasb_hybrid.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\pipeline\segmenters\fusion_hybrid.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\pipeline\detectors\person_detector.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\pipeline\detectors\stub_runner.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\eval\batch.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\eval\calibration.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\eval\experiment.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\config\loader.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\eval\gemini_refine.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\pipeline\segmenters__init__.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\reporting\spec.yaml

7. user (2026-07-05T06:54:09.211Z)

```
83:from backend.storage.capabilities import (  
84:    storage_capabilities, # P3: secret-free "which media can this box use" report  
359:@app.get("/api/capabilities")  
360:def capabilities(request: Request):  
402:    "storage": storage_capabilities(  
726:        from backend.storage.capabilities import _gdrive_mount_root
```

8. user (2026-07-05T06:54:11.713Z)

```
359    @app.get("/api/capabilities")  
360    def capabilities(request: Request):  
361        """Honest probe of what THIS box can do ? the first-run wizard renders the compute  
step from  
362        it (PACKAGING_PLAN.md P1). No secrets: reports only whether a Gemini key is PRESENT,  
never its  
363        value. Best-effort + never raises (the wizard degrades gracefully)."""
```

```

364     cfg = request.app.state.config
365     indexing = getattr(cfg, "indexing", None)
366     deployment = getattr(cfg, "deployment", None)
367     ffmpeg = shutil.which(ffmpeg_bin()) # P2a: honor RALLY_FFMPEG / RALLY_FFPROBE
368     ffprobe = shutil.which(ffprobe_bin())
369     # WASB weights are best-effort: env first (matches native_wasb_runner), then config.
370     wasb = getattr(indexing, "wasb", None)
371     weights_path = (
372         os.environ.get("WASB_WEIGHTS") or getattr(wasb, "weights_path", "") or ""
373     )
374     # Cheap GPU heuristic: nvidia-smi on PATH (matches scripts/doctor.py). Deliberately
NOT
375     # torch.cuda.is_available() ? torch is absent in the LIGHT tier (so it would false-
negative on a
376     # real GPU box) and importing it would add seconds to this probe. The detector
healthcheck is the
377     # authority on whether GPU inference actually works; this is just the wizard's
first-order signal.
378     nvidia_smi = shutil.which("nvidia-smi")
379     return {
380         "api_version": API_VERSION,
381         "engine_version": _engine_version(),
382         "segmenters": sorted(segmenter_registry.list_available().keys()),
383         "default_segmenter": getattr(indexing, "default_segmenter", None),
384         "ffmpeg": {
385             "ffmpeg": bool(ffmpeg),
386             "ffprobe": bool(ffprobe),
387             "ffmpeg_path": ffmpeg,
388             "ffprobe_path": ffprobe,
389         },
390         "gpu": {"nvidia_smi_present": bool(nvidia_smi)},
391         "gemini_key_present": bool(os.environ.get("GEMINI_API_KEY")),
392         "wasb_weights_present": bool(weights_path) and os.path.exists(weights_path),
393         "deployment": {
394             "profile": getattr(deployment, "profile", "") or "",
395             "compute_target": getattr(deployment, "compute_target", "") or "",
396             # item 3: can this server satisfy a per-request compute="cloud" dispatch?
The SPA shows
397             # the "run in the cloud" toggle only when true (else it's local-only).
398             "cloud_available": _cloud_available(cfg),
399         },
400         # P3: which storage media THIS box can store INTO (local always; s3/gcs if
SDK+config;
401         # gdrive if a mount resolves). The P5 medium picker renders only usable
backends. No secrets.
402         "storage": storage_capabilities(
403             getattr(cfg, "storage", None), local_free_path=output_base(cfg)
404         ),
405     }
406
407
408     def _friendly_video_name(filepath: str) -> str:
409         """A human name for a video from its stored path/URI ? the filename stem
(``~/match_2026.mp4`` ?
410         ``match_2026``). Works for a local path, a bucket key, or a ``gdrive://`` URI
(rsplitted on ``/``).
411         Falls back to the whole ``filepath`` if there's no basename."""
412         base = str(filepath or "").rstrip("/").rsplit("/", 1)[-1].rsplit("\\", 1)[-1]
413         stem = os.path.splitext(base)[0]
414         return stem or base or str(filepath or "")
415
416
417     def _video_dims(validation_results: Any) -> "tuple[Optional[float], Optional[str]]":
418         """Pull ``(fps, resolution)`` from the persisted validation results, or ``(None,
None)``. The

```

```

419     ``resolution_fps_check`` validator already probed ffprobe and stored
``details={width,height,fps}``
420     on its result (every branch) ? so the videos row carries it with no schema change /
re-probe. """
421     for r in validation_results or []:
422         if isinstance(r, dict) and r.get("validator_name") == "resolution_fps_check":
423             d = r.get("details") or {}
424             w, h, fps = d.get("width"), d.get("height"), d.get("fps")
425             resolution = f"{w}x{h}" if w and h else None
426             return (fps if isinstance(fps, (int, float)) else None), resolution
427     return None, None
428
429
430     def _video_summary(row: Dict[str, Any]) -> Dict[str, Any]:
431         """Map a DB video row to the API/SPA contract. ADDITIVE ? keeps the raw
``id``/``filepath`` (an
432         existing test + any old client rely on them) and adds the fields the SPA library +
the persistent
433         "current video" label actually read: ``video_id`` (the MD5 join key = ``id``), a
friendly
434         ``match_name`` (so an uploaded video shows its filename, not a generic fallback),
``local_path``,
435         ``created_at`` (from ``processed_at``), and ``fps``/``resolution`` (from the stored
ffprobe
436         validation details) so the library cards can show them. Mirrors the mock engine's
shape so mock and
437         real agree. """
438         fp = row.get("filepath") or ""
439         fps, resolution = _video_dims(row.get("validation_results"))
440         return {
441             **row,
442             "video_id": row.get("id"),
443             "match_name": _friendly_video_name(fp),
444             "local_path": fp,
445             "created_at": row.get("processed_at") or row.get("created_at"),
446             "fps": fps,
447             "resolution": resolution,
448         }
449
450
451     @app.get("/api/videos")
452     def list_videos(request: Request, limit: Optional[int] = None):
453         """List indexed videos (most recent first) so the console survives a reload ?
previously only a
454         single-row get existed (PACKAGING_PLAN.md P1). Rows are enriched to the SPA contract
455         (``video_id`` + a friendly ``match_name`` derived from the filename) so the library
shows real
456         names and the persistent "current video" label survives an upload ? see
``_video_summary``. """
457         return {
458             "videos": [
459                 _video_summary(v) for v in request.app.state.db.list_videos(limit=limit)
460             ]
461         }
462
463
464     def _reels_dir(cfg) -> str:
465         return getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
466
467
468     _REEL_EXTS = (".mp4", ".mov")
469
470
471     @app.get("/api/reels")
472     def list_reels(request: Request):

```

```

473         """List compiled highlight reels present in the output dir, newest first, with a
download URL.
474
475         NB: reels are stored FLAT in output_dir under the bare name passed to /api/compile ?
the engine
476         does NOT track a video_id?reel association today (a per-video listing would be a
phantom), so
477         this is a global list. (Per-video reel tracking would need a DB record + compile
change ? a
478         later slice.) Only top-level video files are listed (per-video scratch subdirs are
skipped)."""

```

9. assistant (2026-07-05T06:54:18.519Z)

So `default\_segmenter` comes from `indexing.default\_segmenter`. Now let me find how that config default resolves, and importantly what `/api/process` does when no segmenter is passed. Let me look at the config defaults and the process handler.

10. user (2026-07-05T06:54:20.414Z)

```

178-
179-
180-class IndexingConfig(BaseModel):
181:     default_segmenter: str = "yolo_hybrid"
182:     use_gpu: bool = True
183:     # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
184:     # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
--
272:     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
273:     fusion: FusionConfig = Field(default_factory=FusionConfig)
274-
275:     @field_validator("default_segmenter")
276:     @classmethod
277:     def segmenter_must_be_registered(cls, v: str) -> str:
278:         # Lazy import: the segmenter registry imports modules that may import

```

11. user (2026-07-05T06:54:21.670Z)

```

63:from backend.pipeline.segmenters.base import segmenter_registry
257:     segmenter: Optional[str] = None
266:     # play_segments is returned from cache so refresh/Try again cannot re-bill paid
segmenters.
324:     (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
332:     "default_segmenter": getattr(indexing, "default_segmenter", None),
382:     "segmenters": sorted(segmenter_registry.list_available().keys()),
383:     "default_segmenter": getattr(indexing, "default_segmenter", None),
675:     Edge-auth mirrors /api/process (default-OFF ? no-op locally). ``video_id`` is validated
as a bare
742:     Edge-auth mirrors ``/api/process``. Exactly one source: an ``uploaded_path`` (a local
file from
898:@app.post("/api/process", status_code=202)
899:def process_video(req: ProcessRequest, request: Request):
902:     Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
953:     if req.segmenter and not segmenter_registry.get(req.segmenter):
954:         available = list(segmenter_registry.list_available().keys())
957:         detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}",
964:         req.segmenter,
1100:     Async, mirroring /api/process: the ffmpeg stitch (trim + re-encode each clip + concat +
watermark)
1313:     available_segmenters = list(segmenter_registry.list_available().keys())
1315:     "--segmenter",
1316:     choices=available_segmenters,
1317:     help=f"Choose rally detection segmenter. Available: {available_segmenters}",

```

```

1520:         seg_name = args.segmenter or getattr(
1521:             getattr(global_config, "indexing", None), "default_segmenter", "motion"
1523:         segmenter_actual = None
1539:         segmenter_cls = segmenter_registry.get(seg_name)
1540:         if not segmenter_cls:
1544:             print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
1545:             segmenter_actual = seg_name
1547:             segmenter = segmenter_cls(db_instance, global_config)
1548:             result = segmenter.process_video(
1576:                 segmenter_requested=args.segmenter,
1577:                 segmenter_actual=segmenter_actual,

```

12. assistant (2026-07-05T06:54:25.299Z)

Confirmed: `IndexingConfig.default\_segmenter = "yolo\_hybrid"` is the code default. The deployed config.json does NOT override `indexing.default\_segmenter` (only sets `skip\_ai\_handoff`), so it stays "yolo_hybrid". Now let me read the process handler in full and the dispatch logic.

13. user (2026-07-05T06:54:26.736Z)

```

898     @app.post("/api/process", status_code=202)
899     def process_video(req: ProcessRequest, request: Request):
900         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
901
902         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
903         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
904         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
905         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
906         """
907         # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
908         # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
909         # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
910         try:
911             owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
912         except edge_auth.EdgeAuthError as e:
913             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
914
915         allowed_root = getattr(
916             getattr(request.app.state.config, "security", None), "allowed_video_root", None
917         )
918         # In hosted mode a bucket URI is allowed only when it's under the configured
input_root (the
919         # designated bucket) ? not an arbitrary bucket the runtime SA happens to reach
(tenant isolation).
920         allowed_uri_root = getattr(
921             getattr(request.app.state.config, "storage", None), "input_root", None
922         )
923         try:
924             validate_input_path(
925                 req.video_path, allowed_root=allowed_root, allowed_uri_root=allowed_uri_root
926             )
927             # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
928             # scheme, which we map to a clean 400 (an unknown scheme must never 500).
929             input_ref = storage.resolve_input_ref(
930                 req.video_path, getattr(request.app.state.config, "storage", None)
931             )
932         except ValueError as e:
933             raise HTTPException(status_code=400, detail=str(e)) from e
934         # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
935         # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI

```

```

string. A
936         # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
937         # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
938         # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
939         # rejects a non-local URI as out-of-root.
940         if input_ref.backend == "local" and not os.path.exists(input_ref.key):
941             raise HTTPException(
942                 status_code=404, detail=f"Video file not found at '{req.video_path}'"
943             )
944         # P2 (D11): a bucket/Drive source is fetched to a scratch copy on the worker; fail
fast with 507
945         # if this box can't hold that copy. Best-effort ? an unstatable remote
(network/SDK) or an
946         # unreadable temp fs skips the guard (measuring must not deny a legit job); the
fetch itself
947         # still fails loudly downstream if space genuinely runs out. Local inputs need no
copy ? skipped.
948         if input_ref.backend != "local":
949             need = _remote_input_size_bytes(
950                 input_ref, getattr(request.app.state.config, "storage", None)
951             )
952             require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
953         if req.segmenter and not segmenter_registry.get(req.segmenter):
954             available = list(segmenter_registry.list_available().keys())
955             raise HTTPException(
956                 status_code=400,
957                 detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}",
958             )
959
960         job_id = uuid.uuid4().hex
961         if not request.app.state.db.create_job(
962             job_id,
963             req.video_path,
964             req.segmenter,
965             req.player_pool,
966             req.max_frames,
967             owner_id=owner_id,
968             locale=req.locale,
969             compute=req.compute,
970             force=req.force,
971         ):
972             raise HTTPException(status_code=500, detail="Failed to persist job record.")
973         _get_executor().submit(
974             _drain_due_jobs, request.app.state.config, request.app.state.db
975         )
976         return JSONResponse(
977             status_code=202,
978             content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
979         )
980
981
982     @app.get("/api/jobs/{job_id}/cost")
983     def get_job_cost(job_id: str, request: Request):
984         """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of
truth (matches
985         GCP billing); ``cost_inr`` is a display conversion at the configured FX rate.
``cost_usd`` is
986         null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
987         cost = request.app.state.db.get_job_cost(job_id)
988         if cost is None:
989             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")

```



```

990     fx = request.app.state.config.billing.fx_inr_per_usd
991     usd = cost.get("cost_usd")
992     cost["cost_inr"] = billing.to_inr(usd, fx) if usd is not None else None
993     cost["fx_inr_per_usd"] = fx
994     return cost
995
996
997     @app.get("/api/videos/{video_id}/cost")
998     def get_video_cost(video_id: str, request: Request):
999         """Aggregate per-video cost across its jobs (M8): ``total_cost_usd`` +
1000         ``total_cost_inr`` + the
1001         per-job rows. A video is 1:1 with an infer job, but a re-ingest can produce
1002         several."""
1003         agg = request.app.state.db.get_video_cost(video_id)
1004         fx = request.app.state.config.billing.fx_inr_per_usd
1005         agg["total_cost_inr"] = billing.to_inr(agg["total_cost_usd"], fx)
1006         agg["fx_inr_per_usd"] = fx
1007         return agg
1008
1009     @app.get("/api/jobs/{job_id}")
1010     def get_job(job_id: str, request: Request):
1011         """Job state: {status, progress, result, reports}. `status` = execution state
1012         (queued|running|done|failed); the pipeline verdict is result["status"]."""
1013         job = request.app.state.db.get_job(job_id)
1014         if not job:
1015             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
1016         result = job.get("result")
1017         return {
1018             "job_id": job["id"],

```

14. assistant (2026-07-05T06:54:35.812Z)

Key finding: the process handler stores `req.segmenter` verbatim into the job (which may be `None`). It does NOT apply the default at this point. So the segmenter default gets resolved later ? either in the worker's job drain or in the dispatch. Let me trace `create\_job` and the drain/dispatch path. Let me look at orchestration.py for the dispatch and segmenter resolution.

15. user (2026-07-05T06:54:37.864Z)

```

4:Cloud Run dispatch + ingest path (`maybe_dispatch_remote` / `_ingest_remote_segments`),
17:Object-attribute patches (`m.storage.fetch`, `m.segmenter_registry.get`, ...) hit the
shared
33:from backend.pipeline.segmenters.base import segmenter_registry
47:# so that raising the drain's worker count (to overlap I/O-bound REMOTE Cloud Run
dispatch/poll) never
49:# rate-fragile; a remote dispatch is a bucket-poll and never holds this lane, so N remote
jobs overlap
111:     persist each rally via the SAME `db.add_play_segment` the in-process segmenters call ?
so
158:     """Whether THIS server can satisfy a per-request `compute="cloud"` dispatch (item 3).
163:     are present so a forced per-request override can actually dispatch + collect a result.
176:def maybe_dispatch_remote(
192:     actually EXECUTED (`False` for a pre-dispatch config rejection, so the observability
report
193:     doesn't claim a fake segmentation run) ? or `None` to fall through to the in-process
segmenter
199:     # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows (id >
play_wm);
203:     # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker rc /
ingest
204:     # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
records the
205:     # intended backend, `requested` the picker choice, `dispatched` whether the remote
job ran.

```

```

217:         "dispatched": ran,
224:     # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch (guarded
by
248:         return None # default-OFF ? run the in-process segmenter below (unchanged)
261:         # can't read it. Stage it UP to the input bucket first, then dispatch from there
(#461). Only
308:         # Repoint BOTH the dispatch and the DB video record at the staged bucket object: the
L4 job
324:     notify("dispatching (cloud_run)")
326:     "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
341:     segmenter=seg_name,
347:     ) # dispatch the Cloud Run Job + bucket-poll to completion
348:     except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job failure,
not a 500
349:         logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
350:         return _failed(f"cloud dispatch error: {e}", True)
394:     "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s video_id=%s",
451:     # consumers (hash / validators / segmenter) use the resolved local path.
472:     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
474:     getattr(config, "indexing", None), "default_segmenter", "motion"
476:     seg_name = req.segmenter or default_seg
477:     segmenter_actual = None
482:     # "in_process" once the local segmenter is reached, set on the response as "cloud_run"
by
483:     # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
538:     # 2. Run segmenter & indexer
544:     segmenter_cls = segmenter_registry.get(seg_name)
545:     if not segmenter_cls:
547:         # in-worker path (e.g. config default pointing at an unregistered segmenter).
548:         available = list(segmenter_registry.list_available().keys())
552:         "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
558:         logger.info(f"[API] Using segmenter: {seg_name}")
560:         segmenter_actual = seg_name
566:         # DEFAULT-OFF: returns None for every non-cloud target ? the in-process segmenter
below runs
569:         remote = _maybe_dispatch_remote(
579:         segmenter = segmenter_cls(db, config)
582:         # dispatch never reaches here (it returned above), so remote polls still overlap
freely.
584:         result = segmenter.process_video(
588:         # A raising segmenter must not leave half-written rows: restore the pre-run
614:         "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.", # type: ignore[arg-type] # hasattr-unwrap above; Success arm has
.value
634:         "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
638:         prov.get("dispatched"),
656:         segmenter_requested=req.segmenter,
657:         segmenter_actual=segmenter_actual,
826:         # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).

```

16. assistant (2026-07-05T06:54:40.810Z)

This is the key. The `default_segmenter` is resolved on line 474-476 inside the worker/drain job (in `orchestration.py`), then passed to `_maybe_dispatch_remote`. Let me read that critical section closely ? lines 470-580.

17. user (2026-07-05T06:54:42.213Z)

```

440         if config is None:
441             config = _main.global_config
442         if db is None:
443             db = _main.db_instance

```

```

444
445     notify = progress or (lambda stage: None)
446
447     notify("hashing")
448     # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
449     # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
450     # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
451     # consumers (hash / validators / segmenter) use the resolved local path.
452     local_video = ""
453     try:
454         local_video = storage.acquire_local_input(
455             req.video_path, getattr(config, "storage", None)
456         )
457         video_id = _main.compute_video_id(local_video)
458     except Exception:
459         storage.cleanup_acquired_local_input(
460             req.video_path, local_video, getattr(config, "storage", None)
461         )
462         raise
463     try:
464         existing_video = db.get_video(video_id)
465     except Exception:
466         storage.cleanup_acquired_local_input(
467             req.video_path, local_video, getattr(config, "storage", None)
468         )
469         raise
470     was_reingest = existing_video is not None
471
472     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
473     default_seg = getattr(
474         getattr(config, "indexing", None), "default_segmenter", "motion"
475     )
476     seg_name = req.segmenter or default_seg
477     segmenter_actual = None
478     segmentation_ran = False
479     val_results: List[Any] = []
480     val_context: Dict[str, Any] = {}
481     # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
482     # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
483     # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
484     compute_actual: Optional[str] = None
485     response: Optional[Dict[str, Any]] = None
486     try:
487         if (
488             existing_video
489             and existing_video.get("status") == "processed"
490             and not req.force
491         ):
492             cached_segments = db.query_play_segments(video_id, {})
493             if cached_segments:
494                 logger.info(
495                     "[API] Reusing cached segmentation for video_id=%s; force=true
bypasses cache.",
496                     video_id,
497                 )
498             response = {
499                 "video_id": video_id,
500                 "status": "success",
501                 "message": (

```

```

502             f"Reused {len(cached_segments)} cached play segments. "
503             "Pass force=true to reprocess this video."
504         ),
505         "results": existing_video.get("validation_results", []),
506         "play_segments": cached_segments,
507         "cached": True,
508         "compute": {
509             "path": "cached",
510             "requested": getattr(req, "compute", None),
511         },
512     }
513     return response
514
515     # 1. Run pluggable validators (with-context variant surfaces ffmpeg_meta for
the report)
516     notify("validating")
517     logger.info(f"Running validations for {req.video_path}...")
518     val_results, val_context = registry.run_all_with_context(local_video, config)
519     failures = [r for r in val_results if not r.passed]
520
521     if failures:
522         # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
523         # status; it no longer destroys the video's prior good segments.
524         db.add_video(
525             video_id,
526             req.video_path,
527             "failed",
528             [r.model_dump() for r in val_results],
529         )
530         response = {
531             "video_id": video_id,
532             "status": "failed",
533             "message": _validation_failure_message(failures),
534             "results": [r.model_dump() for r in val_results],
535         }
536         return response
537
538     # 2. Run segmenter & indexer
539     logger.info("[API] Video passed checks. Segmenting and indexing...")
540     db.add_video(
541         video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
542     )
543
544     segmenter_cls = segmenter_registry.get(seg_name)
545     if not segmenter_cls:
546         # Submit-time validation already 400s unknown names; this is the defensive
547         # in-worker path (e.g. config default pointing at an unregistered
segmenter).
548         available = list(segmenter_registry.list_available().keys())
549         response = {
550             "video_id": video_id,
551             "status": "failed",
552             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
553             "results": [r.model_dump() for r in val_results],
554             "play_segments": [],
555         }
556         return response
557
558     logger.info(f"[API] Using segmenter: {seg_name}")
559     notify(f"segmenting ({seg_name})")
560     segmenter_actual = seg_name
561     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
562     # if the run yields segments; otherwise drop the new partial rows and keep the

```

```

old.
563         play_wm, cand_wm = db.segment_watermarks(video_id)
564         # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
565         # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
566         # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
567         # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
568         # they're dropped on the cloud path in this slice ? a documented follow-up.)
569         remote = _maybe_dispatch_remote(
570             config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
571         )
572         if remote is not None:
573             response, segmentation_ran = (
574                 remote # `ran` = did the cloud job actually execute (report)
575             )
576             return response # the cloud branch already stamped response["compute"]
(path=cloud_run)
577         # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
578         compute_actual = "in_process"
579         segmenter = segmenter_cls(db, config)
580         try:
581             # Serialize the CPU/GPU-heavy in-process detection (see
_LOCAL_COMPUTE_LANE). A remote
582             # dispatch never reaches here (it returned above), so remote polls still
overlap freely.
583             with _LOCAL_COMPUTE_LANE:
584                 result = segmenter.process_video(
585                     video_id, local_video, req.player_pool, req.max_frames
586                 )
587         except Exception:
588             # A raising segmenter must not leave half-written rows: restore the pre-run
589             # state (same destroy-after-confirm contract as the Failure branch), then
let

```

18. assistant (2026-07-05T06:54:49.886Z)

This confirms the CP-side resolution: `seg\_name = req.segmenter or default\_seg` where `default\_seg = indexing.default\_segmenter` = "yolo_hybrid" (deployed). This `seg\_name` is passed to `\_maybe\_dispatch\_remote(..., seg\_name, ...)` . Now the critical question: does the dispatch forward `seg\_name` to the worker job, and what does the worker default to? Let me read `\_maybe\_dispatch\_remote` fully (lines 176-400).

19. user (2026-07-05T06:54:51.263Z)

```

176     def _maybe_dispatch_remote(
177         cfg,
178         db,
179         req: "ProcessRequest",
180         video_id: str,
181         seg_name: str,
182         val_results,
183         play_wm: int,
184         cand_wm: int,
185         notify,
186     ) -> Optional[Tuple[Dict[str, Any], bool]]:
187         """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
`deployment.compute_target
188         == "cloud_run"` , run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
intervals into
189         THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
190

```

```

191         Returns `` (response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether
the cloud job
192         actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
observability report
193         doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-
process segmenter
194         (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every
non-``cloud_run`` target, so
195         the in-process path stays byte-identical)."""
196         from backend.compute import ComputeJobSpec, get_compute_backend
197
198         def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
199             # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
(id > play_wm);
200             # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
201             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
202             # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
only when the
203             # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
rc / ingest
204             # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
records the
205             # intended backend, `requested` the picker choice, `dispatched` whether the
remote job ran.
206             return (
207                 {
208                     "video_id": video_id,
209                     "status": "failed",
210                     "message": msg,
211                     "results": [r.model_dump() for r in val_results],
212                     "play_segments": [],
213                     "compute": {
214                         "path": "cloud_run" if ran else "not_run",
215                         "requested": req.compute,
216                         "target": "cloud_run",
217                         "dispatched": ran,
218                     },
219                 },
220                 ran,
221             )
222
223             # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
224             # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
225             # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
226             choice = (req.compute or "").strip().lower()
227             if choice and choice not in ("local", "cloud"):
228                 logger.warning(
229                     "[API] ignoring unknown compute=%r (expected 'local'/'cloud'); using server
default.",
230                     req.compute,
231                 )
232             choice = ""
233             if choice == "local":
234                 return None # explicit "run on my machine" ? in-process regardless of the
server's target
235             if choice == "cloud":
236                 if not _cloud_available(cfg):
237                     return _failed(
238                         "cloud-serving was requested but this server isn't configured for it "
239                         "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
240                         False,

```

```

241         )
242         compute = get_compute_backend(cfg, target_override="cloud_run")
243     else:
244         compute = get_compute_backend(
245             cfg
246         ) # server default (default-OFF unless cloud_run)
247     if compute is None:
248         return None # default-OFF ? run the in-process segmenter below (unchanged)
249
250     storage_cfg = getattr(cfg, "storage", None)
251
252     # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
253     # to this box.
254     try:
255         input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
256     except ValueError as e:
257         return _failed(
258             f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
259         )
260     if input_ref.backend == "local":
261         # A browser upload lands on THIS box's local disk; the Cloud Run job (a
262         # different machine)
263         # can't read it. Stage it UP to the input bucket first, then dispatch from there
264         (#461). Only
265         # possible when a CLOUD input_root is configured; without one we genuinely can't
266         # stage ? fail.
267         # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged
268         # URI below is
269         # built by joining under input_root, and storage.put routes by
270         # parse_uri(staged_uri).backend ?
271         # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left
272         # at its
273         # default, while a bare/local-looking root can't stage no matter what
274         # input_backend says
275         # (resolve_input_uri only consults input_backend when input_root is EMPTY, and
276         # an empty root
277         # gives us nothing to build the staged URI under).
278         input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
279         try:
280             root_backend = (
281                 storage.parse_uri(input_root).backend if input_root else "local"
282             )
283         except ValueError: # unsupported scheme in input_root ? nowhere real to stage
284             # to
285             root_backend = "local"
286         if root_backend == "local":
287             return _failed(
288                 "cloud-serving needs a cloud input (e.g. gs://?), or a cloud
289                 storage.input_root to "
290                 "stage local uploads into; a path local to this machine can't be read by
291                 the Cloud "
292                 "Run job.",
293                 False,
294             )
295     local_src = input_ref.key # resolved local path of the upload
296     staged_uri =
297     f"{input_root.rstrip('/')}/{req.video_id}/{os.path.basename(local_src)}"
298     try:
299         notify("staging upload to bucket")
300         # Thread the active storage settings so backend-specific config survives (S3
301         endpoint_url/
302         # region/addressing for R2·D0·MinIO; GCS project/client) ? not a fresh
303         default client.
304         storage.put(
305             local_src,

```

```

291         staged_uri,
292         config=(
293             storage_cfg.model_dump() if storage_cfg is not None else None
294         ),
295     )
296     except Exception as e: # noqa: BLE001 ? a staging failure is a clean job
failure, not a 500
297         logger.error(
298             "[API] cloud-serving: failed to stage %s -> %s: %s",
299             local_src,
300             staged_uri,
301             e,
302             exc_info=True,
303         )
304         return _failed(
305             f"cloud-serving: failed to stage the uploaded file to {input_root}:
{e}",
306             False,
307         )
308         # Repoint BOTH the dispatch and the DB video record at the staged bucket object:
the L4 job
309         # DETECTs from it, and a later /api/compile re-fetches the same source (the
local upload is
310         # gone once this instance recycles). add_video is an UPSERT ? it only updates
filepath.
311         req.video_path = staged_uri
312         input_ref = storage.resolve_input_ref(staged_uri, storage_cfg)
313         db.add_video(
314             video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
315         )
316         out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
317         if not out_root:
318             return _failed(
319                 "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
320                 "? that's where the Cloud Run job writes the result.",
321                 False,
322             )
323
324         notify("dispatching (cloud_run)")
325         logger.info(
326             "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
327             req.video_path,
328             out_root,
329         )
330         # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
331         # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
332         # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
333         # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
334         overrides = []
335         sk = getattr(getattr(cfg, "indexing", None), "skip_ai_handoff", None)
336         if sk is not None:
337             overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
338         spec = ComputeJobSpec(
339             video_uri=req.video_path,
340             out_uri=out_root,
341             segmenter=seg_name,
342             config_overrides=tuple(overrides),
343         )
344         try:
345             result = compute.run_infer(

```



```

346         spec
347     ) # dispatch the Cloud Run Job + bucket-poll to completion
348 except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
349     logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
350     return _failed(f"cloud dispatch error: {e}", True)
351 if result.returncode != 0:
352     return _failed(f"cloud job failed (returncode {result.returncode}).", True)
353 if result.video_id and result.video_id != video_id:
354     # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
355     # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
356     logger.warning(
357         "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
358         "and the worker hash; ingesting under the local id.",
359         result.video_id,
360         video_id,
361     )
362
363     notify("ingesting (cloud_run)")
364     storage_dict = (
365         storage_cfg.model_dump() # type: ignore[union-attr] # hasattr-guarded; mypy
can't narrow
366         if hasattr(storage_cfg, "model_dump")
367         else {}
368     )
369     try:
370         _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
371 except Exception as e: # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
372     logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
373     return _failed(
374         f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
375         True,
376     )
377
378     # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
379     segments = [
380         s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
381     ]
382     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
383     # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
384     # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
385     provenance = {
386         "path": "cloud_run",
387         "requested": req.compute,
388         "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
389         "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
390         "execution": result.execution,
391         "outputs_uri": result.outputs_uri,
392     }
393     logger.info(
394         "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
395         result.execution,

```

20. assistant (2026-07-05T06:54:59.796Z)

Critical confirmation at lines 338-343: `ComputeJobSpec(... segmenter=seg_name, ...)`. The `seg_name` computed on the CP (which is `req.segmenter` or `default_seg` = "yolo_hybrid" when SPA

omits segmenter) IS forwarded to the worker. So the worker is told explicitly which segmenter to use. Now I need to verify:

1. That the CP forwards `seg\_name` as an explicit `--segmenter` arg to the worker (so the worker default doesn't matter).
2. Whether the config_overrides forwards the segmenter (it does NOT ? only skip_ai_handoff).

Let me check how ComputeJobSpec.segmenter becomes the worker's `--segmenter` arg, and what the worker defaults to. Let me look at the compute backend that builds the job args.

21. user (2026-07-05T06:55:01.762Z)

```
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\compute\__init__.py:15:from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\compute\__init__.py:20:    "ComputeJobSpec",
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\compute\base.py:21:class ComputeJobSpec:
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\compute\base.py:28:    segmenter: Optional[str] = None
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\compute\base.py:36:    (0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 = remote dispatch timeout or an
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\compute\base.py:51:    """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns only once
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\compute\base.py:56:    def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\compute\cloud_run.py:35:from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\compute\cloud_run.py:171:    # None ? a fresh uuid4 per run_infer call
(prod); injected ? deterministic (tests).
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\compute\cloud_run.py:179:    def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\compute\cloud_run.py:192:        if spec.segmenter:
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\compute\cloud_run.py:193:            argv += ["--segmenter", spec.segmenter]
```

22. assistant (2026-07-05T06:55:08.441Z)

Line 192-193: `if spec.segmenter: argv += ["--segmenter", spec.segmenter]`. Since `seg\_name` is always truthy ("yolo_hybrid" from the default when SPA omits it), the worker gets `--segmenter yolo\_hybrid` explicitly. Let me read the cloud_run.py argv construction fully and then confirm the worker's `\_\_main\_\_.py` uses that arg (and what it defaults to if absent).

23. user (2026-07-05T06:55:09.280Z)

```
179         def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
180             out_root = spec.out_uri.rstrip("/")
181             token = self._token or uuid.uuid4().hex
182             done_uri = f"{out_root}/_dispatch/{token}.done"
183             argv: List[str] = [
184                 "infer",
185                 "--video-uri",
186                 spec.video_uri,
187                 "--out-uri",
188                 spec.out_uri,
189                 "--done-uri",
190                 done_uri,
191             ]
192             if spec.segmenter:
```

```

193         argv += ["--segmenter", spec.segmenter]
194     for kv in (*spec.config_overrides, *self._container_overrides):
195         argv += ["--set", kv]
196
197     execution = self._client.run_job(
198         self._job_name, argv, region=self._region, project=self._project
199     )
200     LOG.info(
201         "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
202         self._job_name,
203         execution,
204         done_uri,
205     )
206
207     try:
208         marker = poll_until(
209             lambda: self._read_marker(done_uri),
210             self._policy,
211             label="cloud-run-infer",
212             logger=LOG,
213         )
214     except PolicyTimeout as timeout_exc:
215         marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
216         video_id = str(marker.get("video_id", ""))
217         # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
218         # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
219         rc = int(marker.get("returncode", 1))
220         if not video_id:
221             LOG.warning(
222                 "cloud-run: job=%s completion marker present but empty/unreadable (no "
223                 "video_id) ? treating as failure",

```

24. assistant (2026-07-05T06:55:12.081Z)

Now let me confirm the worker `\_\_main\_\_.py` uses `--segmenter` and what it defaults to if the arg were absent (to be thorough about the fallback claim).

25. user (2026-07-05T06:55:13.567Z)

```

4:         [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
14:(validator registry ? segmenter ? report), never a forked pipeline.
38:from backend.pipeline.segmenters.base import segmenter_registry
147:     segmenter=args.segmenter,
246:     segmenter_actual = None
256:     seg_name = args.segmenter or config.indexing.default_segmenter
257:     segmenter_cls = segmenter_registry.get(seg_name)
258:     if not segmenter_cls:
260:         f"[worker] unknown segmenter: {seg_name!r} "
261:         f"(have {list(segmenter_registry.list_available())})"
264:     segmenter_actual = seg_name
265:     result = segmenter_cls(db, config).process_video(
270:         print(f"[worker] segmenter FAILED: {result.message}")
284:         "infer produced 0 segments for video_id=%s (segmenter=%s, detector_impl=%s).
"
287:         "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the key
or "
293:         segmenter_actual,
305:         segmenter_requested=args.segmenter,
306:         segmenter_actual=segmenter_actual,
422:     Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build the
423:     detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo) has
426:     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
428:     seg_cls = segmenter_registry.get(args.segmenter)

```

```

431:         f"[worker] unknown segmenter: {args.segmenter!r} "
432:         f"(have {list(segmenter_registry.list_available())})"
438:         f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
453:     print(f"[worker] detector ready ({args.segmenter}): {msg}")
633:     p.add_argument(
642:     pi.add_argument(
645:     pi.add_argument(
650:     pi.add_argument(
651:         "--segmenter", default=None, help="default: indexing.default_segmenter"
653:     pi.add_argument(
656:     pi.add_argument(
659:     pi.add_argument("--max-frames", type=int, default=None)
660:     pi.add_argument(
667:     pi.add_argument(
680:     pt.add_argument(
686:     pt.add_argument(
691:     pt.add_argument("--version", required=True, help="semver for the artifact bundle")
692:     pt.add_argument(
700:     pt.add_argument(
701:         "--segmenter",
705:     pt.add_argument(
708:     pt.add_argument("--config", default=None)
709:     pt.add_argument("--scratch", default=None)
710:     pt.add_argument(
716:     pt.add_argument(
722:     pt.add_argument("--set", action="append", default=[], metavar="k=v")
729:     VISIBLE ? the worker previously ran with no logging config, so every segmenter/detector
log

```

26. assistant (2026-07-05T06:55:23.123Z)

This is the decisive line. Worker line 256: ``seg_name = args.segmenter or config.indexing.default_segmenter``.

- ``--segmenter`` defaults to ``None`` (line 651).
- The CP always passes ``--segmenter yolo_hybrid`` explicitly (because ``seg_name`` on the CP was resolved to "yolo_hybrid" and ``spec.segmenter`` is truthy ? argv gets ``--segmenter yolo_hybrid``).
- So ``args.segmenter`` = "yolo_hybrid" on the worker ? the worker's own ``config.indexing.default_segmenter`` fallback is NEVER reached.

The worker's own default is irrelevant. The CP-resolved value wins. Let me confirm the SPA side to complete the chain ? does the SPA send ``segmenter`` at all?

27. user (2026-07-05T06:55:25.773Z)

```

web/src/api:
client.test.ts
client.ts
locale.test.ts
types.ts
upload.test.ts

```

```

web/src/components:
a4.touch.test.tsx
a8.status.test.tsx
AnnotatePanel.test.tsx
AnnotatePanel.tsx
build-error.test.tsx
CompileResult.tsx
FfmpegBanner.test.tsx
FfmpegBanner.tsx
IngestPanel.test.tsx
IngestPanel.tsx
LanguageSwitcher.tsx
QueryBar.examples.test.ts

```

```

QueryBar.test.tsx
QueryBar.tsx
RallyGrid.tsx
SelectionFooter.test.tsx
SelectionFooter.tsx
SetupWizard.test.tsx
SetupWizard.tsx
StatusChip.tsx
TapFilters.test.tsx
TapFilters.tsx
upload.test.tsx
VideoLibrary.test.tsx
VideoLibrary.tsx
---
web/src/api/types.ts:20:  /** motion-segmenter FABRICATION (random) ? DO NOT surface (see
PER_RALLY_SELECTION.md). */
web/src/api/types.ts:22:  /** motion-segmenter FABRICATION (random) ? DO NOT surface. */
web/src/api/types.ts:95:  segmenters?: string[];
web/src/api/types.ts:96:  default_segmenter?: string;
web/src/components/SetupWizard.test.tsx:24:  default_segmenter: "motion",
web/src/components/SetupWizard.test.tsx:30:  default_segmenter: "gemini",
web/src/components/SetupWizard.test.tsx:38:  default_segmenter: "wasb_hybrid",
web/src/components/SetupWizard.tsx:54:  const detector = caps?.default_segmenter ?? "motion";

```

28. assistant (2026-07-05T06:55:32.318Z)

The grep shows `segmenter` appears only in `types.ts` (capabilities shape) and `SetupWizard.tsx` (reads caps for display). It does NOT appear in `client.ts` or `IngestPanel.tsx`. Let me confirm the `processVideo` call in client.ts and the IngestPanel to be definitive that no segmenter is sent.

29. user (2026-07-05T06:55:33.669Z)

```

66:      body: opts.body,
77:      log("error", "api.http_error", { cid, method, url, status: res.status, body:
body.slice(0, 500) });
87: *   processing. POST /api/process {video_path} ? 202 {job_id, ...} [engine process route].
Fast-fails 400
89: export async function processVideo(
93:   compute?: string,
95:   // `locale` (the UI language) is recorded for PMF analytics (which markets use the tool).
`compute`
96:   // is the compute-picker choice ("local" | "cloud") ? where DETECT runs; omitted ? the
server
97:   // default. Both optional + omitted when absent (JSON.stringify drops undefined).
100:   body: JSON.stringify({ video_path: videoPath, locale, compute }),
128:   body: JSON.stringify({ video_id: videoId, csv }),
137: *   at rest, registered as-is). Feed `stored_uri` into processVideo(). Fast-fails 400 (bad
medium /
146:   body: JSON.stringify({
160:   body: JSON.stringify({ video_id: videoId }),
187:   body: JSON.stringify({ video_id: videoId, segment_ids: segmentIds, output_filename:
outputFilename, locale }),
207:     throw new ApiError(msg, { cid, status: 502, body: msg });
213:     throw new ApiError(msg, { cid, status: 502, body: msg });
216:     throw new ApiError("Compile timed out.", { cid, status: 504, body: "Compile timed
out." });
245:// bodies ~100 MB), then `complete` returns a server-host `path` to feed straight into
/api/process.
251:   body: JSON.stringify({ filename, total_size_bytes: totalSizeBytes }),
271:   body: chunk,
281:   log("error", "api.http_error", { cid: c, method: "PUT", url, status: res.status, body:
body.slice(0, 500) });
288:/** Finalize: returns {path, video_id} ? feed `path` to processVideo()
[backend/api/uploads.py]. */

```

```
292:   body: JSON.stringify({ total_parts: totalParts } ),
```

30. assistant (2026-07-05T06:55:41.511Z)

Definitive. Line 100: `body: JSON.stringify({ video\_path: videoPath, locale, compute })`. The processVideo call sends ONLY `video\_path`, `locale`, and `compute`. There is NO `segmenter` field. `JSON.stringify` drops undefined, so `req.segmenter` arrives as `None` on the engine.

Let me quickly confirm IngestPanel calls processVideo with only compute (no segmenter picker), to close the SPA side fully.

31. user (2026-07-05T06:55:42.754Z)

```
58: // Compute-picker (PACKAGING compute-picker, item 3): if this server can dispatch to a
cloud GPU
62: const [compute, setCompute] = useState<Compute>("local");
91: const sendCompute = cloudAvailable ? compute : undefined;
94: const cloudBlocked = cloudAvailable && compute === "cloud" && !cloudAck;
214:   <fieldset className="mt-4 rounded-xl border border-black/10 p-3" data-
testid="compute-picker">
216:     {t("ingest.compute.legend")}
223:     aria-pressed={compute === opt}
228:     (compute === opt
233:     {t(`ingest.compute.${opt}`)}
237:   {compute === "cloud" && (
238:     <div className="mt-3" data-testid="compute-cost">
239:       <p className="text-xs text-ink/60">{t("ingest.compute.cloudCost")}</p>
248:       {t("ingest.compute.ack")}
251:       <p className="mt-1 text-xs text-amber-700" data-testid="compute-ack-
required">
252:         {t("ingest.compute.ackRequired")}
466:       <Trans i18nKey="errors.fileTooLargeHelp.computer" components={{ b:
<strong /> }} />
```

32. assistant (2026-07-05T06:55:50.273Z)

Confirmed. IngestPanel has a compute-picker (local/cloud toggle) but NO segmenter picker. Let me confirm the exact processVideo invocation in IngestPanel (which args it passes).

33. user (2026-07-05T06:55:52.361Z)

(Bash completed with no output)

34. user (2026-07-05T06:55:59.401Z)

```
web/src/components/IngestPanel.tsx:91: const sendCompute = cloudAvailable ? compute :
undefined;
web/src/components/IngestPanel.tsx:116:   submit(asset.stored_uri, locale, sendCompute,
displayName ?? sourceLabel(asset.stored_uri) ?? undefined); // index the stored copy
web/src/components/IngestPanel.tsx:127: // (useUpload reads onCompleteRef.current), so
`medium`/`mediumUri`/`sendCompute` are always fresh.
web/src/components/IngestPanel.tsx:129:   if (medium === "local") submit(path, locale,
sendCompute, filename);
web/src/components/IngestPanel.tsx:149:   submit(uri, locale, sendCompute, displayName ??
undefined);
web/src/api/client.ts:89:export async function processVideo(
web/src/api/client.ts:137: * at rest, registered as-is). Feed `stored_uri` into processVideo().
Fast-fails 400 (bad medium /
web/src/api/client.ts:288:/** Finalize: returns {path, video_id} ? feed `path` to processVideo()
[backend/api/uploads.py]. */
web/src/api/types.ts:140: /** The durable, MD5-keyed location ? pass as video_path to
processVideo(). */
web/src/hooks/useIngestJob.ts:10:import { getJob, processVideo } from "../api/client";
web/src/hooks/useIngestJob.ts:259:   const resp = await processVideo(uri, cid, locale,
```

```
compute);
```

35. assistant (2026-07-05T06:56:03.781Z)

The chain is fully confirmed. `useIngestJob.ts:259` calls `processVideo(uri, cid, locale, compute)` ? the 4 positional args match client.ts signature `(videoPath, cid, locale, compute)`. No segmenter is ever passed. `submit` threads `sendCompute` (which is the local/cloud choice or undefined) as the compute value.

I have traced the complete chain end-to-end across all three components. Let me record the config default validation once more to be certain there's no config.json override for the segmenter, then finalize.

36. user (2026-07-05T06:56:04.838Z)

```
180     class IndexingConfig(BaseModel):
181         default_segmenter: str = "yolo_hybrid"
182         use_gpu: bool = True
183         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
184         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
185         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
186         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
187         # RALLY_DETECTOR_IMPL env var overrides this. See
188         pipeline/detectors/{stub,native_wasb}_runner.py.
189         detector_impl: str = "auto"
190         motion_threshold: float = 0.08
```

37. assistant (2026-07-05T06:56:08.104Z)

I have all the evidence needed. The trace is complete and definitive across all three components.

38. user (2026-07-05T06:56:46.115Z)

Structured output provided successfully